

Design and Analysis of Algorithms

What is an algorithm?

- An algorithm is a finite set of instructions or logic, written in order, to accomplish a certain predefined task.
- Algorithm is not the complete code or program, it is just the core logic(solution) of a problem
- can be expressed either as an informal high level description as **pseudo code** or using a **flowchart**.

Characteristics of an Algorithm

- **Input-** There should be 0 or more inputs supplied externally to the algorithm.
- **Output-** There should be at least 1 output obtained.
- **Definiteness-** Every step of the algorithm should be clear (Unambiguous) and well defined.
- **Finiteness-** The algorithm should terminates after a finite number of steps.
- **Effectiveness-** Every instruction must be basic enough so that it can be carried out.

Four Steps in Problem Solving

1. Design an Algorithm
2. Validate an Algorithm
3. Analyse an Algorithm
4. Test a program

In this subject we will focus on **Design** and **Analysis** of an algorithm

Design of an Algorithm

- It is an art which may never be fully automated.
- There are number of design techniques.
- The techniques, which will be discussed in this subject are:
 - Divide and Conquer
 - Greedy Technique
 - Dynamic Programming
 - Backtracking
 - Branch and Bound

Analysis of an Algorithm

- Analysis of algorithms or performance analysis refers to the task of determining how much **computing time** and **storage** an algorithm requires.
- It allows to make quantitative comparison of performance of different algorithms.
- It allows to predict performance of an algorithm under different situations like best case, worst case and average case.

Complexity

- **Space Complexity:** It is the amount of memory an algorithm needs to run to completion
- **Time Complexity:** It is the amount of computer time an algorithm needs to run to completion

Space Complexity

- The space needed is the sum of **Fixed** space and **Variable** space
- **Fixed space: c**
 - Includes the instructions, variables, and constants
 - Independent of the number and size of I/O
- **Variable space: $S_p(I)$**
 - Includes dynamic allocation, functions' recursion
- Total space of any program
 - **$S(P) = c + S_p(\text{Instance})$**

Space Complexity

- Example 1:

```
Algorithm abc(a, b, c)  
{  
    return  $a + b + b * c + (a + b - c) / (a + b) + 4.0$ ;  
}
```

$$S_{abc}(I) = 0$$

Space Complexity

- Example 2:

```
Algorithm Sum( $a, n$ )  
{  
     $s := 0.0$ ;  
    for  $i := 1$  to  $n$  do  
         $s := s + a[i]$ ;  
    return  $s$ ;  
}
```

$S_{\text{Sum}}(n) \geq (n + 3)$ (n for $a[]$, one each for n , i , and s)

Space Complexity

- Example 3:

```
Algorithm RSum( $a, n$ )  
{  
    if ( $n \leq 0$ ) then return 0.0;  
    else return RSum( $a, n - 1$ ) +  $a[n]$ ;  
}
```

The recursion stack space needed $\geq 3(n+1)$

Time Complexity

- The total time = Compile Time + Run Time
 - $T(P) = c + T_p(\text{Instance})$
- A compiled program will be run several times without recompilation. Consequently, we concern ourselves with just the run time of a program i.e. $T_p(\text{Instance})$
- **Program Step:** A program step is a syntactically or semantically meaningful program segment whose execution time is independent of the instance characteristics

Methods to compute the step count

- Introduce variable count into programs
- Tabular method
 - Determine the total number of steps contributed by each statement
 - $\text{steps_per_statement} * \text{frequency_for_that_statement}$
 - s/e, steps/execution
 - add up the contribution of all statements

Using ***count*** variable

- Example 1:

```
Algorithm Sum(a, n)
{
    s := 0.0;
    count := count + 1; // count is global; it is initially zero.
    for i := 1 to n do
    {
        count := count + 1; // For for
        s := s + a[i]; count := count + 1; // For assignment
    }
    count := count + 1; // For last time of for
    count := count + 1; // For the return
    return s;
}
```

$2n + 3$ steps

Using ***count*** variable

- Example 2: (Simplified version of Example 1)

```
Algorithm Sum(a, n)  
{  
    for i := 1 to n do count := count + 2;  
    count := count + 3;  
}
```

$2n + 3$ steps

Using *count* variable

- Example 3:

```
Algorithm RSum(a, n)
{
    count := count + 1; // For the if conditional
    if (n ≤ 0) then
    {
        count := count + 1; // For the return
        return 0.0;
    }
    else
    {
        count := count + 1; // For the addition, function
                           // invocation and return
        return RSum(a, n - 1) + a[n];
    }
}
```

$$t_{\text{RSum}}(n) = \begin{cases} 2 & \text{if } n = 0 \\ 2 + t_{\text{RSum}}(n - 1) & \text{if } n > 0 \end{cases}$$

Using *count* variable

- Example 3:

$$\begin{aligned}t_{\text{RSum}}(n) &= 2 + t_{\text{RSum}}(n-1) \\&= 2 + 2 + t_{\text{RSum}}(n-2) \\&= 2(2) + t_{\text{RSum}}(n-2) \\&\vdots \\&= n(2) + t_{\text{RSum}}(0) \\&= 2n + 2, \qquad n \geq 0\end{aligned}$$

2n + 2 steps

Tabular Method

- Example 1:

Statement	s/e	frequency	total steps
1 Algorithm Sum(a, n)	0	—	0
2 {	0	—	0
3 $s := 0.0;$	1	1	1
4 for $i := 1$ to n do	1	$n + 1$	$n + 1$
5 $s := s + a[i];$	1	n	n
6 return $s;$	1	1	1
7 }	0	—	0
Total			$2n + 3$

Tabular Method

■ Example 2:

Statement	s/e	frequency		total steps	
		$n = 0$	$n > 0$	$n = 0$	$n > 0$
1 Algorithm RSum(a, n)	0	—	—	0	0
2 {					
3 if ($n \leq 0$) then	1	1	1	1	1
4 return 0.0;	1	1	0	1	0
5 else return					
6 RSum($a, n - 1$) + $a[n]$;	$1 + x$	0	1	0	$1 + x$
7 }	0	—	—	0	0
Total				2	$2 + x$

$$x = t_{\text{RSum}}(n - 1)$$

Exercise 1

- Find the step counts for the following algorithm of “Matrix addition” using both methods

Algorithm Add(a, b, c, m, n)
{
 for $i := 1$ **to** m **do**
 for $j := 1$ **to** n **do**
 $c[i, j] := a[i, j] + b[i, j];$
}

Exercise 1 : Solution

- Using **count** variable

```

Algorithm Add( $a, b, c, m, n$ )
{
  for  $i := 1$  to  $m$  do
  {
     $count := count + 1$ ; // For ‘for  $i$ ’
    for  $j := 1$  to  $n$  do
    {
       $count := count + 1$ ; // For ‘for  $j$ ’
       $c[i, j] := a[i, j] + b[i, j]$ ;
       $count := count + 1$ ; // For the assignment
    }
     $count := count + 1$ ; // For loop initialization and
                        // last time of ‘for  $j$ ’
  }
   $count := count + 1$ ; // For loop initialization and
                        // last time of ‘for  $i$ ’
}

```

Exercise 1 : Solution

- Using **Tabular** method

Statement	s/e	frequency	total steps
1 Algorithm Add(a, b, c, m, n)	0	—	0
2 {	0	—	0
3 for $i := 1$ to m do	1	$m + 1$	$m + 1$
4 for $j := 1$ to n do	1	$m(n + 1)$	$mn + m$
5 $c[i, j] := a[i, j] + b[i, j];$	1	mn	mn
6 }	0	—	0
Total			$2mn + 2m + 1$

Time Complexity

- The time complexity of an algorithm is given by the number of steps taken by the algorithm to compute the function it was written for.
- The number of steps is itself a function of the instance characteristics.
- Although any specific instance may have several characteristics (e.g., the number of inputs, the number of outputs, the magnitudes of the inputs and outputs).
- The number of steps is computed as a function of some subset of these

Time Complexity

- For many algorithms, the time complexity is not dependent solely on the number of inputs or outputs or some other easily specified characteristic.
- For example, the searching algorithm.
- It may terminate in one step if x is the first element examined by algorithm, or it may take two steps (this happens if x is the second element examined), and so on.
- In other words, knowing n alone is not enough to estimate the run time of the algorithm.

3 kinds of step counts

- When the chosen parameters are not adequate to determine the step count uniquely by defining three kinds of step counts: **best case**, **worst case** and **average**.
- The **best-case** step count is the minimum number of steps that can be executed for the given parameters.
- The **worst-case** step count is the maximum number of steps that can be executed for the given parameters.
- The **average** step count is the average number of steps executed on instances with the given parameters.

Difficulty with Step Counts

- Determining the exact step count (best case, worst case, or average) of an algorithm can prove to be an exceedingly difficult task.
- It is not worthy to put immense efforts to determine the step count exactly, since the notion of a step is itself inexact.
- E.g. Both the instructions
 1. $x := y;$
 2. $x := y + z + (x/y) + (x * y * z - x/z);$count as one step

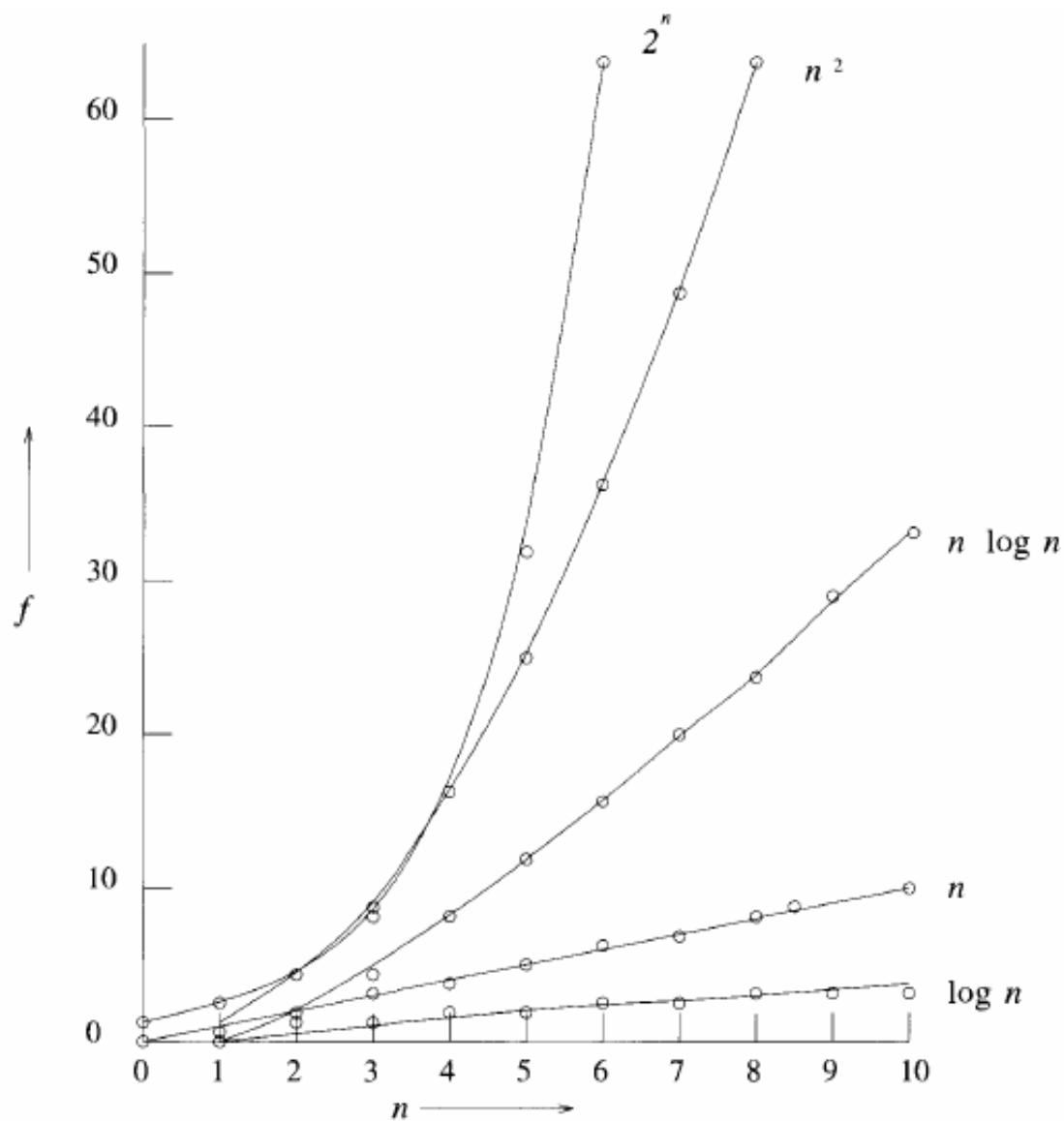
Asymptotic Notation

- As it is very difficult to determine exact step count, Asymptotic Notations are used to measure complexities.
- It enables to make meaningful (but inexact) statements about the time and space complexities of an algorithm
- Asymptotic Notations describe complexity in terms of some function of instance characteristic (generally denoted with n)
- For examples: n , $\log(n)$, $n \cdot \log(n)$, n^2 , n^3 , 2^n .

Function Values

$\log n$	n	$n \log n$	n^2	n^3	2^n
0	1	0	1	1	2
1	2	2	4	8	4
2	4	8	16	64	16
3	8	24	64	512	256
4	16	64	256	4,096	65,536
5	32	160	1,024	32,768	4,294,967,296

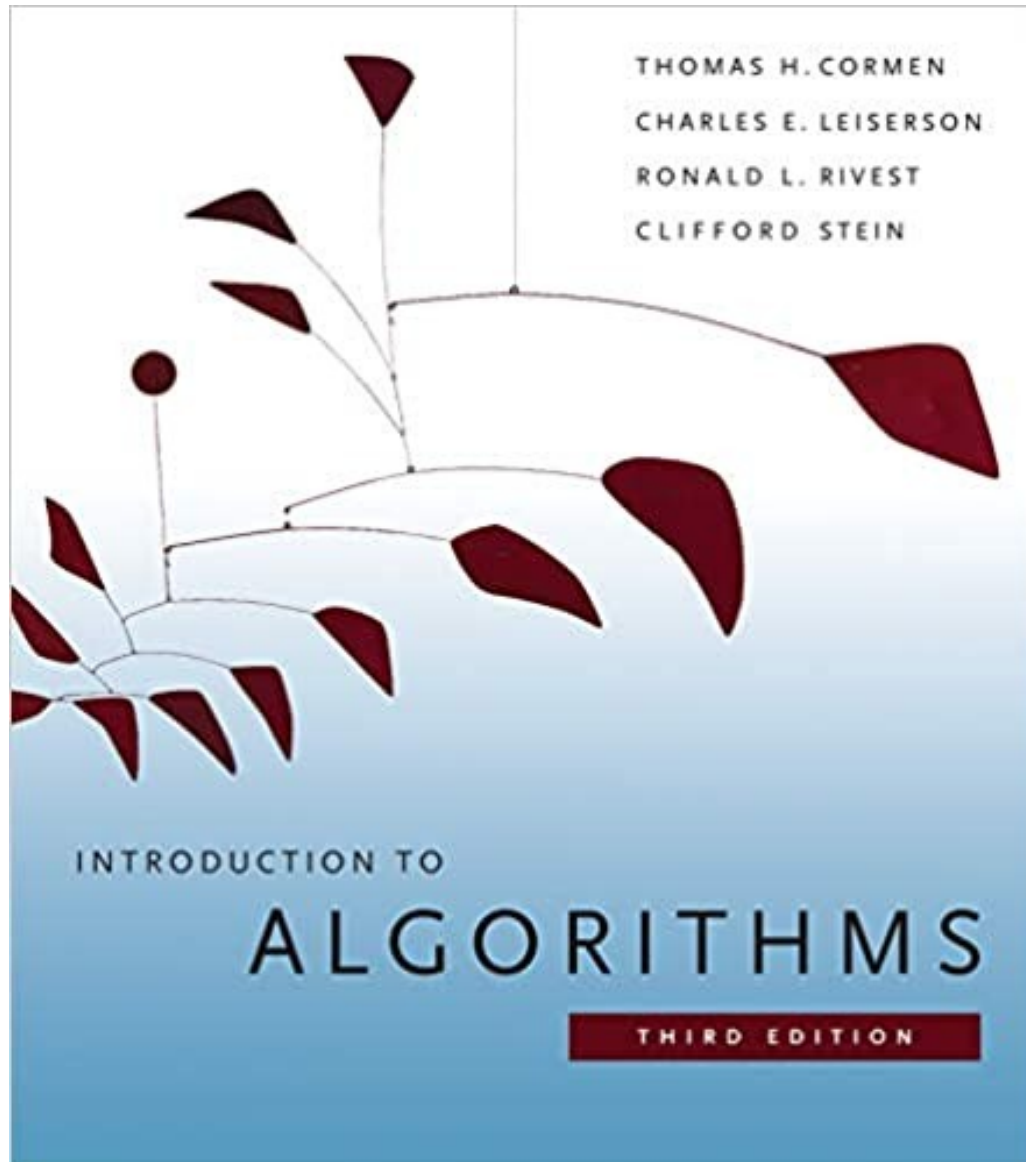
Plot of Function Values



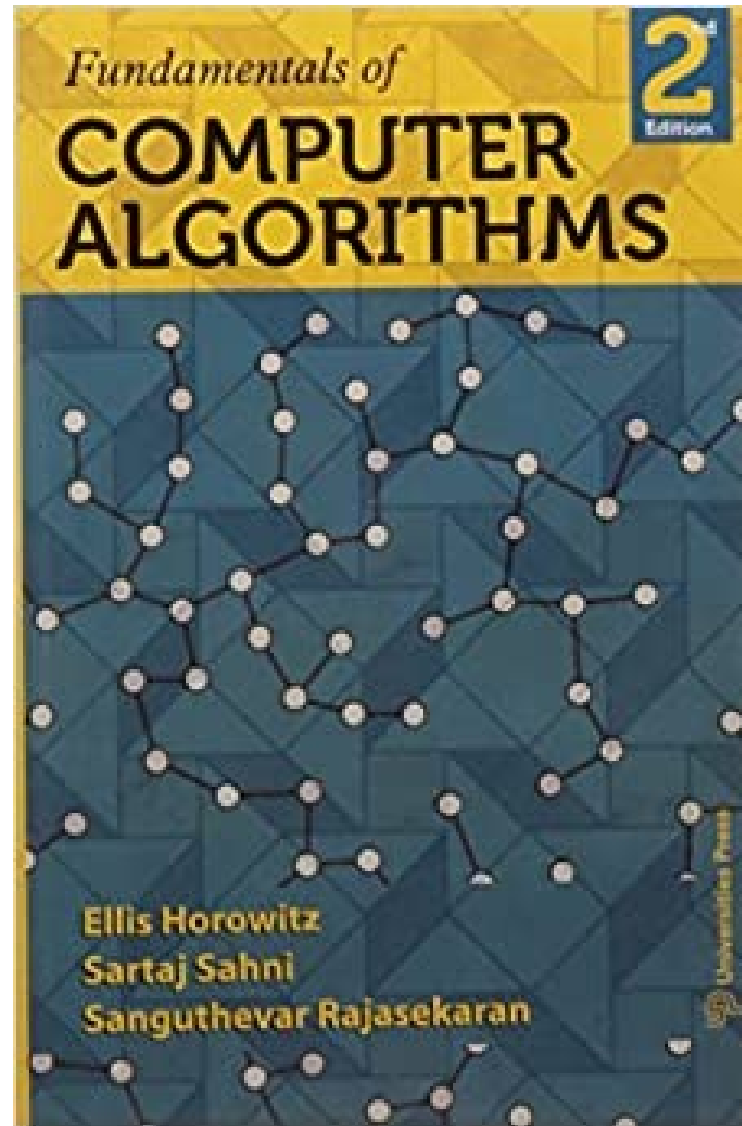
References

1. A Book – “**Fundamentals of Computer Algorithms**” by Horowitz, Sahni, Rajasekaran

Text & Reference Books



Text & Reference Books



Text & Reference Books

